

Node Maintenance & Scaling

Kubernetes Production · Lesson 16

Universal `kubectl` maintenance + Kubespray Day-2 scaling

`kubernetes.io/docs` · `kubespray.io`

Learning Objectives

By the end of this lesson you will be able to:

- Take a node out of service with **cordon / drain / uncordon**
- Use drain flags `--ignore-daemonsets`, `--delete-emptydir-data`, `--force`
- Protect availability during eviction with a **PodDisruptionBudget**
- Explain **graceful termination** and `terminationGracePeriodSeconds`
- Add a worker on Kubespray with `scale.yml` (+ `--limit`)
- Change cluster membership: `remove-node.yml`, control-plane & etcd changes

Framing: `drain` is *how you safely evict*; Kubespray's `scale.yml` / `remove-node.yml` are *how you change cluster membership*.

16.1 Cordon, Drain, Uncordon

Cordon, Drain, Uncordon

The three verbs of node maintenance — **universal**, work on any cluster.

Command	What it does
<code>kubectl cordon <node></code>	Marks the node unschedulable (<code>node.kubernetes.io/unschedulable</code> taint). No new Pods land; existing Pods stay.
<code>kubectl drain <node></code>	Safely evicts eligible Pods with graceful termination, respecting PDBs . Auto-cordons the node first.
<code>kubectl uncordon <node></code>	Re-enables scheduling after maintenance.

- You rarely call `cordon` alone — `drain` cordons for you.
- Once `drain` returns **without errors**, it is safe to power down or delete the node.

Always **uncordon** when maintenance is finished, or the node stays empty.

Draining a Node

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

Flag	Effect
<code>--ignore-daemonsets</code>	Proceed despite DaemonSet Pods. Required when any exist — they're re-created instantly and tolerate the taint, so they can't be evicted. Without it, drain fails .
<code>--delete-emptydir-data</code>	Allow eviction of Pods using local <code>emptyDir</code> storage (that data is lost).
<code>--force</code>	Evict/delete bare Pods not managed by a controller. They won't be rescheduled — use with care.

`--force` does **not** bypass graceful termination — `terminationGracePeriodSeconds` still applies to every Pod.

What Drain Evicts and Respects

- **Evicted** — regular app Pods and Pods owned by **Deployments, ReplicaSets, StatefulSets, Jobs**. Their controllers reschedule replacements on other nodes.
- **Ignored (not moved):**
 - **DaemonSet** Pods (with `--ignore-daemonsets`).
 - **Mirror / static Pods** — bound to the node via `nodeName`.
 - Any Pod with `nodeName` **set directly** — it keeps running on the drained node.

```
# Verify the node is empty of evictable Pods afterwards
kubectl get pods -A -o wide --field-selector spec.nodeName=<node-name>
```

CKA tip: if drain **hangs**, a **PodDisruptionBudget** is blocking it — drain *waits* rather than break the budget (next section).

16.2 Graceful Termination

Graceful Termination

When a Pod is evicted or deleted, the kubelet shuts it down in **stages**:

1. Send **SIGTERM** to the container's main process.
2. Wait up to **terminationGracePeriodSeconds** (default **30s**).
3. If still running, send **SIGKILL** (forceful kill).

```
spec:
  terminationGracePeriodSeconds: 60
  containers:
  - name: app
    image: nginx
    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "nginx -s quit; sleep 5"]
```

- The container should **catch SIGTERM** to finish in-flight work, flush, close.
- A **preStop** hook runs **before** SIGTERM — useful to drain LB connections.

16.3 PodDisruptionBudget

PodDisruptionBudget (PDB)

A **PDB** caps how many Pods of a set may be down at once; drain's **Eviction API** honors it.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2           # or: maxUnavailable: 1
  selector:
    matchLabels:
      app: web
```

- Set **minAvailable** or **maxUnavailable** (number or percentage).
- An eviction that would breach the budget is **blocked**; drain **waits**.
- Guards **voluntary** disruptions only (drain, Eviction API) — **not** a node crash, kernel panic or hardware failure.

PDB During Drain & Concurrent Drains

- During `drain`, evictions go through the **Eviction API** (`/eviction` subresource), which checks the PDB before removing each Pod.
- Drain **waits** for evicted Pods to come back healthy elsewhere before evicting the next — keeping the budget satisfied throughout.
- **Concurrent drains** of several nodes still respect each PDB **collectively**:

A StatefulSet of **3** replicas with `minAvailable: 2` lets only **1** Pod be evicted at a time — across **all** running drains combined.

- The Eviction API gives this PDB-respecting behavior **programmatically**, without `kubect1 drain`.

16.4 Kubespray — Adding a Worker

Adding a Worker — `scale.yml`

On Kubespray you change membership by editing the **inventory** and running the **matching playbook** — idempotent, touches only what changed.

```
# 0. Refresh facts for ALL nodes first (avoids stale-facts bugs with --limit)
ansible-playbook -i inventory/mycluster/hosts.yaml facts.yml

# 1. Add the new host to the inventory, then:
ansible-playbook -i inventory/mycluster/hosts.yaml scale.yml --limit=node4
```

- `scale.yml` is the **additive worker path** — joins new workers without re-running the whole cluster.
- `--limit=NODE_NAME` scopes the run to the new node only.
- **Facts caveat:** run `facts.yml` without `--limit` first, or the limited run works from **stale facts** about the rest of the cluster.

You **cannot** use `scale.yml` for a **control-plane** node (next section).

16.5 Kubespray — Control-Plane Changes

Adding a Control-Plane Node — `cluster.yml`

A control-plane node is **not** added with `scale.yml`.

```
# Append the new host to the END of kube_control_plane, then:  
ansible-playbook -i inventory/mycluster/hosts.yaml cluster.yml
```

- **Always append** new control-plane nodes to the **end** of the `kube_control_plane` group. Adding one in the **first** position is **not supported** and the playbook **fails**.
- After the run, **restart `nginx-proxy` on every host** so it reloads config:

```
crictl ps | grep nginx-proxy | awk '{print $1}' | xargs crictl stop
```

`cluster.yml` re-issues apiserver/cert config across the control plane, wiring the new member into certs, kubeconfig and the apiserver SAN/endpoint list.

etcd Members & the First-Node Caveat

- **Keep etcd odd** (3, 5, ...) for quorum — adding etcd is a **replace** or a **scale-up by two**. Add via `cluster.yml` + `upgrade-cluster.yml` with `--limit=etcd,kube_control_plane -e ignore_assert_errors=yes`, then add the new member to `--etcd-servers=...` in each `kube-apiserver.yml`.
- **First-node caveat** — you **cannot directly remove the first** node in `kube_control_plane` / etcd:

1. **Reorder** the inventory so the first entry moves back (e.g. `node-2`, `node-3`, `node-1`).
2. Run `cluster.yml` / `upgrade-cluster.yml` to apply the new ordering.
3. *Then* remove the old first node with `remove-node.yml`.

16.6 Kubespray — Removing a Node

remove-node.yml — Cordon, Drain, Reset

The single entry point for removing **any** node type — worker, control-plane or etcd.

```
# Node still in the inventory; drains + resets it:
ansible-playbook -i inventory/mycluster/hosts.yaml remove-node.yml \
  -e node=node4

# Offline / unreachable node — skip the on-node reset:
ansible-playbook -i inventory/mycluster/hosts.yaml remove-node.yml \
  -e node=node4 -e reset_nodes=false -e allow_ungraceful_removal=true
```

- By default it **cordons** → **drains** → **resets** (wipes the K8s install/config).
- **-e node=NODE_NAME** is mandatory; comma-separate multiple (**node4,node5**).
- Run it **while the node is still in the inventory**, *then* delete it from the inventory.

Kubespray's playbooks **wrap the same kubectl drain** you run by hand — safe eviction underpins every membership change.

Key Takeaways

- **cordon / drain / uncordon** = take a node out and back; always **uncordon** after
- `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data` is the canonical command
- `--ignore-daemonsets` required if any DaemonSet Pods exist; `--force` only for bare Pods
- **Graceful termination** = SIGTERM → `terminationGracePeriodSeconds` (30s) → SIGKILL
- A **PDB** blocks evictions that breach `minAvailable` / `maxUnavailable`; drain **waits**
- Kubespray worker → `scale.yml` (+ `--limit`, prior `facts.yml`); control-plane → `cluster.yml`
- Any removal → `remove-node.yml -e node=NODE_NAME` (drains + resets); append CP nodes to the **end**

End of Lesson 16

Next: Lesson 17 — Upgrades

```
kubectl drain --ignore-daemonsets --delete-emptydir-data · kubectl uncordon · scale.yml --limit ·  
remove-node.yml -e node=
```